Food Truck Live

Manual de instalação

Guia completo, do início aos sites publicados. Dados, imagens e emails, passo a passo.

Guia ilustrado | 3 de novembro de 2026



Índice

1. Escolher a instalação 3

2. Iniciador universal — Windows, macOS e Linux 4

3. Criar o sistema e explorar a demo 5

4. Docker local — macOS e Linux 6

5. Servidor Node.js ou Docker público 7

6. Vercel 8

7. Cloudflare Workers — sites com API externa 9

8. Demo web e variáveis privadas 10

9. Backup, restauro e atualização 11

10. Diagnóstico e validação 12

1. Escolher a instalação

Este pacote contém dois sites, uma API Node.js e PostgreSQL. Escolha instalação local em Windows/macOS/Linux, Docker local, servidor Docker, Vercel ou Cloudflare híbrida. As receitas web requerem configuração técnica e contas próprias. Não incluem serviços contratados nem publicam na sua conta automaticamente.

A demo e o sistema real são instalações independentes. A demo permite apenas consulta; nunca introduza nela dados de clientes reais.

2. Iniciador universal — Windows, macOS e Linux

Descarregue INICIAR.mjs e Food_Truck_Live-1.0.1.zip para a mesma pasta gravável. São os únicos dois ficheiros exteriores da entrega. A aplicação, recursos demo, manuais, código e manifesto estão todos no ZIP.

Instale Node.js 24 LTS com npm a partir de https://nodejs.org. Abra Terminal/PowerShell no Windows ou Terminal no macOS/Linux. Execute:

```
node INICIAR.mjs
```

Não é necessário descompactar manualmente. O iniciador verifica o SHA-256 do arquivo e todos os ficheiros do manifesto antes de instalar. Escolha 1 para abrir a demo ou 2 para criar/abrir o seu sistema. Na primeira utilização descarrega dependências e prepara os sites. Não precisa de Docker, Git, Python nem de instalar PostgreSQL manualmente.

O código fica em .foodtruck/releases, numa pasta própria por versão. Os dados ficam em .foodtruck/data/system; a demo em .foodtruck/data/demo. Esta pasta é oculta em macOS/Linux. Não a elimine nem a distribua. Portas: clientes 3000, gestão 3001, API 4000, PostgreSQL 5433 no sistema ou 5434 na demo. Os serviços só escutam na máquina local.

Compatibilidade prevista: Windows x64; macOS Intel x64 e Apple Silicon ARM64; Linux x64/ARM64 com glibc. Utilize uma conta normal, sem sudo/root. Alpine/musl e Windows ARM nativo não fazem parte deste perfil. Os testes nativos desta entrega foram efetuados em Windows; macOS/Linux permanecem por validar em hardware real.

Mantenha o terminal aberto e termine com Ctrl+C. Os dados são preservados. Se faltar uma ferramenta para abrir o navegador automaticamente, use os endereços apresentados no terminal. Não copie dependências instaladas nem ficheiros de base PostgreSQL entre sistemas operativos.

3. Criar o sistema e explorar a demo

Num sistema novo, o navegador abre uma ligação privada de configuração. Tem validade de uma hora. Preencha organização, primeiro negócio, email e palavra-passe com pelo menos 12 caracteres. A criação é atómica: em caso de erro, pode tentar novamente sem ficar uma configuração parcial.

Depois de concluir, entre na gestão. Configurações posteriores abrem o início de sessão. Um sistema já configurado não pode ser reclamado por outro utilizador nem reiniciado pelo assistente.

Na demo, o acesso de consulta é automático. A faixa permanente identifica o modo. Pode explorar menus, inventário, encomendas fictícias e calendário. Edição, uploads, registos, encomendas e email estão desativados. Para criar o seu sistema, termine o lançador e escolha 2.

4. Docker local — macOS e Linux

Instale Docker Engine/Desktop com Compose v2 e Node.js 24 para os comandos de configuração. Na pasta extraída:

```
npm run install:init -- --profile=docker-local
```

Edite .env.install. Defina POSTGRES_PASSWORD com um segredo aleatório de letras e números. Para o perfil local mantenha os endereços localhost. Gere a ligação privada:

```
npm run install:token
docker compose --env-file .env.install -f compose.local.yaml up -d --build
```

Abra a ligação mostrada por install:token. Sites em http://localhost:3000 e http://localhost:3001. Não precisa de domínio nem certificado. O PostgreSQL fica na rede privada Docker. Os volumes persistem ao terminar:

```
docker compose --env-file .env.install -f compose.local.yaml down
```

Para a demo, utilize uma segunda cópia da configuração e defina FOODTRUCK_PROFILE=demo, DEMO_MODE=1 e ALLOW_DEMO_SEED=1 em .env.install. Termine primeiro o perfil real porque os sites usam as mesmas portas. O nome do projeto cria volumes separados. Abra http://localhost:3001/demo. Nunca use os volumes reais para semear a demo. Não execute down -v: elimina volumes.

5. Servidor Node.js ou Docker público

Configure PostgreSQL, domínios HTTPS, email e armazenamento persistente. No servidor Node.js:

```
npm ci --include=dev
npm run install:init
```

Edite .env.install com DATABASE_URL, DATABASE_MIGRATION_URL se necessária, CUSTOMER_APP_URL, MANAGEMENT_APP_URL, INTERNAL_API_URL, CORS_ORIGINS e DATA_DIRECTORY absoluto. Não desative validação TLS da base. Use COOKIE_SECURE=1 em HTTPS e um CRON_SECRET aleatório com pelo menos 32 caracteres.

```
npm run install:check
npm run db:migrate
npm run install:token
npm run build:install
npm start
```

O arranque Node.js executa os dois sites, API e worker. Use um gestor de processos e proxy inverso para HTTPS. O assistente está em /setup; use apenas a ligação privada fornecida.

Alternativamente configure POSTGRES_PASSWORD, CUSTOMER_DOMAIN e MANAGEMENT_DOMAIN e execute:

```
docker compose --env-file .env.install -f compose.install.yaml up -d --build
```

Para este perfil Docker, a ligação de migração aponta para a mesma base privada do contentor. Não use a base cloud de outra instalação. O perfil servidor usa Caddy e portas 80/443. Garanta DNS correto e diretórios/volumes persistentes. Se usar Cloudflare como proxy DNS, mantenha TLS Full (strict).

6. Vercel

Pode gerar o exemplo com npm run install:init -- --profile=vercel. Crie três projetos a partir da sua cópia do código: apps/customer, apps/management e services/api. Os dois primeiros usam Next.js. A API usa a configuração vercel.json incluída. Use PostgreSQL externo e bucket privado Supabase ou S3/R2 para imagens; o disco de uma função não é armazenamento persistente.

Na API configure as ligações da base, CORS_ORIGINS, CUSTOMER_APP_URL, MANAGEMENT_APP_URL, COOKIE_SECURE=1, API_UPDATES_MODE=poll, serviço de email, storage e CRON_SECRET. Nos sites configure INTERNAL_API_URL para o endereço HTTPS da API, também no ambiente de build. Nunca coloque chaves da base ou do storage no código enviado ao navegador.

Execute as migrações uma vez com ligação direta/session à base correta. Gere install:token localmente, copie INSTALL_TOKEN e INSTALL_TOKEN_EXPIRES_AT apenas para as variáveis privadas da API e publique-a novamente antes de abrir a ligação privada. Concluída a configuração, remova estas duas variáveis. Não execute bootstrap numa atualização.

O agendamento incluído chama /api/v1/internal/maintenance a cada minuto e exige Authorization: Bearer com CRON_SECRET. Escolha um plano que permita esta frequência ou um agendador externo compatível. Verifique os registos de execução e teste a expiração de encomendas; não depende de visitas ao site para funcionar.

Fonte: https://vercel.com/docs/cron-jobs/manage-cron-jobs

7. Cloudflare Workers — sites com API externa

Pode gerar o exemplo com npm run install:init -- --profile=cloudflare. Este perfil publica apenas os sites. A API continua em Node.js/Vercel ou Docker e PostgreSQL continua externo. Não substitua PostgreSQL por D1.

```
npm run cloudflare:prepare -- customer https://api.example.com
npm run cloudflare:prepare -- management https://api.example.com
npm run cf:build --workspace=food-truck-customer
npm run cf:preview --workspace=food-truck-customer
```

Repita build/preview para food-truck-management. Configure INTERNAL_API_URL também no ambiente de build, domínios dos sites e CORS na API. Só depois de testar publique com cf:deploy no workspace escolhido. O adaptador é instalado apenas quando escolhe este percurso. A receita fixa OpenNext 1.20.15 e Wrangler 4.157.0; ambos os sites passaram build e pré-visualização local com Next.js 16.4.3. Em Windows o adaptador recomenda WSL para publicação. Não foi anunciado suporte validado sem build, preview e teste numa conta de staging.

Fonte: https://opennext.js.org/cloudflare

8. Demo web e variáveis privadas

Publique a demo em projetos próprios, com base e storage independentes. Depois das migrações, defina DEMO_MODE=1 e ALLOW_DEMO_SEED=1 apenas nessa instalação e execute npm run seed. O seed recusa bases com dados existentes e preserva uma demo já criada. Depois retire ALLOW_DEMO_SEED.

Abra /demo no site de gestão para acesso de consulta. Não configure fornecedores de email na demo. O bloqueio de alterações existe na API e a manutenção não modifica os dados demonstrativos.

Nunca distribua .env.install, profile.env, chaves de fornecedores, dados reais ou backups. O ZIP inclui apenas ficheiros de exemplo sem segredos.

9. Backup, restauro e atualização

Faça uma cópia antes de atualizar. Na instalação universal, termine o iniciador e copie toda a pasta .foodtruck/data/system para um local seguro. Restaure essa cópia noutra pasta juntamente com a mesma versão da aplicação; só abra uma instância de cada vez. A cópia da pasta PostgreSQL só é válida com os serviços terminados.

Em servidores, use npm run backup com pg_dump da versão compatível, configurando DATABASE_MIGRATION_URL, BACKUP_SCHEMA quando necessário e certificado TLS. Em perfis Windows use FOODTRUCK_ENV_FILE com o caminho absoluto de .foodtruck/data/system/profile.env. O backup inclui imagens locais; objetos Supabase/S3/R2 exigem cópia separada, preservando as chaves uploads/.

Restaure primeiro numa base vazia independente usando pg_restore com --no-owner --no-acl, reponha imagens e execute diagnóstico. Compare contagens e teste login e encomendas antes de substituir uma instalação. Credenciais são fornecidas pelo ambiente privado, nunca no histórico do terminal.

Para Docker pode executar pg_dump/pg_restore no serviço postgres e copiar o volume de imagens. Não faça cópia de ficheiros PostgreSQL com a base em execução.

Para atualizar o percurso universal, substitua ZIP e INICIAR.mjs juntos na mesma pasta, preservando .foodtruck. Para uma instalação web, extraia atualizações numa pasta separada, preserve a configuração, faça backup e teste migrações numa cópia dos dados. Não volte a semear nem a inicializar o administrador. Se for necessário reverter uma migração, restaure o backup na versão anterior; não há downgrade automático.

10. Diagnóstico e validação

Use install:check e install:doctor no perfil web. O lançador Windows explica falta de Node.js, permissões, dependências e portas ocupadas. Se interromper a primeira instalação, volte a abrir o lançador; os passos concluídos e dados existentes são preservados.

Consulte docs/distribution/VALIDACAO.md para resultados desta versão e limitações de ambiente. npm run test:distribution utiliza uma base própria e testa configuração, concorrência, demo e manutenção. npm run test:release verifica o pacote e os hashes. Os testes históricos SQLite não validam a instalação PostgreSQL.